

Smart Garden

Ben Smith, Jiana Kambo

Project overview

Smart Garden will be an automated, connected device for monitoring the conditions of and automatically watering a plant. It will be a system that can track environmental factors including temperature, humidity, soil moisture, and more in order to automate plant watering. It will also inform the user if they need to refill the reservoir and display graphs showing trends for all data. We use an Arduino Uno for all real-time processing while our Lampi will mostly serve as a front-end which sends requests to the Arduino for the latest measurements or activates the pump. There is also a Django web application that allows remote access to the system, shows detailed information from the sensors, and allows the user to adjust the moisture threshold for automatic watering.

Project Plan

After the midterm presentation we understood what parts of the project would be difficult: communication from the Pi to Arduino and from EC2 to the Pi. We were unable to receive data on the Arduino from the Pi at the mid-point report, and we had yet to set up the web application. To finish, we made a list of things we needed to achieve and blocked out the weeks until our presentation date. Our rough timeline was:

Week of April 9 - Establish bi-directional serial communication between the Arduino and Pi.

Week of April 15 - Establish communication between the Pi to the AWS to show real time data

Week of April 22 - Establish communication between AWS to Pi, and data visualization

Week of April 29 - Troubleshoot and write report

Process

We went in order. We first made sure the Arduino and Pi were communicating bi-directionally. Once we got that down we started with getting the Pi to talk to the EC2 website - just sending and displaying the data. We then worked on getting the EC2 website to talk to the Pi - sending the preset values and turning the pump on and off properly. We then tried to make sure the whole architecture is working properly - the Pi and Arduino are talking bidirectionally and the EC2 server is also bi-talking to the Pi. We ran into errors here because we were trying to use two different scripts to communicate with the serial port, but only one script can use the port at a time. We started to troubleshoot here, and solved it by combining both scripts into one file that uses the port and handles communication with both the Arduino and EC2 simultaneously.

We then worked on developing the data visualization, given that this was the last thing we worked on we did not have a lot of data to be collected. Hence in order to properly demonstrate the functionality of the graphs we are displaying data points from every 3 minutes over the last 2 hours.

Results

We completed what we set out to complete. We got our architecture complete with the Pi talking to the Arduino and the website bi-directionally, and we were able to turn the pump on and off from both the website and Pi as well as set preset values on the website.

On the pi we decided to only show the essential information, such as if the plant is above the needed moisture level - if the plant is happy or not - what the moisture level is, what the reservoir level is, and if it is connected to the Arduino. The user is also able to manually trigger the pump from the Pi UI.

On the website we show all sensor values including soil moisture level, the reservoir level, the light level, the temperature, and the humidity. It also lets you set the preset values for reservoir level warning and the threshold for soil moisture level for auto-watering. If these values are not met there is an attention flag shown, so the user can give it the needed attention. Moreover we have the data visualization for all the elements, implemented so we are able to see the trends for all the different factors well to keep the user informed.

Conclusions

We learned how a lot of things work through trial and error, such as the error with the Pi trying to communicate with the website and with the Arduino through the one serial port. We also learned a lot about API endpoints as we decided that since we have only one device and bi directional communication it is better for us to use API endpoints, so we were also able to learn this new way of communication.

We hoped we did baseline work for the data visualization so we could collect more data rather than working on that in the end, but we were able to troubleshoot and still graph data by collecting data every 3 minutes over a few hours to graph it and see the visualization.

We learned a lot in this project. We applied our knowledge from the class, but also researched how to communicate with an Arduino over the serial port and use API endpoints with Django.